

Gold-Guided Critical Learning for Learning and Unlearning from Mixed Data

Zeyu Han

1 Method Formulation

We consider a classification model M_θ parameterized by θ , where the model maps an input sample x to a predictive probability distribution over labels:

$$M_\theta(x) = p_\theta(y \mid x).$$

We assume access to two datasets. The first one is a high-quality gold dataset

$$D_G = \{(x_i^G, y_i^G)\}_{i=1}^{n_G},$$

where each label is considered reliable. The second one is a mixed training dataset

$$D_t = \{(x_i, y_i)\}_{i=1}^{n_t},$$

which may contain both clean samples and noisy or harmful samples. Our goal is to use the reliable knowledge from D_G to guide the learning process on D_t , so that the model can learn from reliable samples while suppressing or correcting misleading signals.

1.1 Gold-Guided Reference Training

We first train the model on the gold dataset D_G to obtain a gold-trained parameter θ_G . Since the labels in D_G are assumed to be reliable, the resulting model captures an initial reliable knowledge state learned from high-quality data. To make the role of this stage explicit, we denote the gold reference training objective as

$$L_{\text{ref}}(\theta; D_G) = \frac{1}{|D_G|} \sum_{(x_i^G, y_i^G) \in D_G} \ell(M_\theta(x_i^G), y_i^G),$$

where $\ell(\cdot, \cdot)$ denotes the supervised classification loss, such as the cross-entropy loss. The gold-trained parameter is then obtained by

$$\theta_G = \arg \min_{\theta} L_{\text{ref}}(\theta; D_G).$$

The gold-trained parameter θ_G plays two roles in the subsequent learning process. First, it provides the initial parameter for the learner trained on the mixed dataset:

$$\theta_0 \leftarrow \theta_G.$$

This means that the model starts mixed-data training from a knowledge state learned from reliable examples, rather than from a randomly initialized or purely mixed-data-trained state.

Second, we keep a frozen copy of the gold-trained model as a reference evaluator:

$$\bar{\theta}_G \leftarrow \theta_G, \quad \bar{\theta}_G \text{ is frozen during mixed-data training.}$$

During the later training stage, $M_{\bar{\theta}_G}$ is not updated. Instead, it is used to evaluate whether each sample in the mixed training dataset D_t is consistent with the reliable knowledge learned from D_G .

In this sense, the gold-trained model is not only a warm-up initialization, but also a gold-guided reference evaluator for judging the reliability of later training signals. This design is motivated by a human learning analogy. Knowledge learned from reliable early education can provide both a foundation for later learning and a criterion for judging whether new information is reasonable. Similarly, the gold-trained parameter initializes the learner, while its frozen copy serves as a stable evaluator for the mixed training data.

1.2 Gold-Consistency Score

For each sample $(x_i, y_i) \in D_t$, we use the frozen gold reference model $M_{\bar{\theta}_G}$ to estimate how consistent the observed label y_i is with the reliable knowledge learned from D_G . In this stage, we do not perform label correction. Instead, the goal is to evaluate the reliability of the given training signal (x_i, y_i) and use this reliability score to control how strongly the learner should learn from it.

We compute the gold-consistency score from two complementary perspectives: prediction-level consistency and representation-level consistency.

First, we define the prediction-level consistency score as

$$r_i^{\text{prob}} = p_{\bar{\theta}_G}(y_i \mid x_i),$$

where $p_{\bar{\theta}_G}(y_i \mid x_i)$ is the probability assigned by the frozen gold reference model to the observed training label y_i . A larger value of r_i^{prob} indicates that the observed label is strongly supported by the predictive distribution of the gold reference model.

Second, we introduce a representation-level similarity score. Let $h_{\bar{\theta}_G}(x)$ denote the feature representation extracted by the frozen gold reference model before the final classification layer. For each class y , we compute a gold feature prototype using the gold samples from that class:

$$\mu_y^G = \frac{1}{|D_G^y|} \sum_{(x_j^G, y_j^G) \in D_G^y} h_{\bar{\theta}_G}(x_j^G),$$

where

$$D_G^y = \{(x_j^G, y_j^G) \in D_G \mid y_j^G = y\}.$$

For the mixed training sample (x_i, y_i) , we compare its feature representation with the gold prototype corresponding to its observed label y_i :

$$s_i = \frac{h_{\bar{\theta}_G}(x_i)^\top \mu_{y_i}^G}{\|h_{\bar{\theta}_G}(x_i)\| \|\mu_{y_i}^G\|}.$$

Since cosine similarity lies in $[-1, 1]$, we normalize it into $[0, 1]$:

$$r_i^{\text{sim}} = \frac{s_i + 1}{2}.$$

The final gold-consistency score is defined as

$$r_i = \beta r_i^{\text{prob}} + (1 - \beta) r_i^{\text{sim}},$$

where $\beta \in [0, 1]$ controls the relative contribution of prediction-level and representation-level consistency.

A larger value of r_i indicates that the observed label y_i is more consistent with the reliable feature-label knowledge learned from the gold dataset. In contrast, a smaller value of r_i suggests that the observed training signal may be unreliable because of label noise, distributional inconsistency, harmful supervision, or uncertainty from the gold evaluator.

Importantly, the role of r_i is not to infer a corrected label for x_i . Instead, it serves as an auxiliary reliability signal that helps the learner decide how strongly it should learn from the current observed training pair (x_i, y_i) . When r_i is high, the observed training pair is treated as likely consistent with the reliable gold knowledge and can contribute a stronger positive learning signal. When r_i is low, the observed training pair is treated as likely contaminated, mislabeled, or unreliable, and its influence should be reduced or suppressed by the later critical controller.

The frozen gold reference model $M_{\bar{\theta}_G}$ is used only as a fixed evaluator. The learner being updated during mixed-data training is M_{θ_k} , whose parameter changes over training iterations.

1.3 Elastic Critical Controller

After obtaining the gold-consistency score r_i , we use an elastic critical controller to transform this reliability signal into a sample-wise learning control signal. Instead of making a binary clean/noisy decision, the controller continuously modulates how strongly the learner should respond to each observed training pair (x_i, y_i) .

The controller is defined as

$$c_i = \phi(r_i),$$

where c_i controls the direction and strength of the sample-wise update. Since $r_i \in [0, 1]$, the controller should map the reliability score into a bounded control signal that can support learning, suppression, or anti-fitting behavior.

We use a nonlinear elastic controller based on the hyperbolic tangent function:

$$c_i = \tanh(\alpha(r_i - \tau)),$$

where τ is a reliability threshold and α controls the sharpness of the transition around this threshold.

The use of $\tanh(\cdot)$ is motivated by three properties. First, it provides a signed output in $[-1, 1]$, which allows the controller to represent positive learning, near-zero suppression, and negative anti-fitting signals within a single continuous formulation. Second, it is smooth around the threshold τ , so samples near the uncertain region are not abruptly classified as either reliable or unreliable. Third, it is bounded, which prevents highly reliable or highly unreliable samples from producing unbounded gradient scaling and helps maintain training stability.

The reliability threshold τ determines the point at which the controller becomes neutral:

$$r_i = \tau \quad \Rightarrow \quad c_i = 0.$$

When $r_i > \tau$, the controller becomes positive and the learner is encouraged to learn from the sample. When $r_i \approx \tau$, the controller is close to zero and the sample is largely suppressed. When $r_i < \tau$, the controller becomes negative, which discourages the learner from fitting the observed label. This negative control should be interpreted as an anti-fitting signal against unreliable supervision, rather than as explicit label correction.

Therefore, the controller has the following interpretation:

$$c_i > 0 \Rightarrow \text{learning from the sample,}$$

$$c_i \approx 0 \Rightarrow \text{suppressing the sample,}$$

$$c_i < 0 \Rightarrow \text{discouraging fitting to the observed label.}$$

In the basic formulation, α is treated as a fixed hyperparameter. A smaller α produces a softer transition around τ , which avoids overly aggressive suppression of uncertain samples. A larger α makes the controller closer to a hard reliability decision, causing the control signal to change more sharply when r_i crosses the threshold.

A possible extension is to use a dynamic sharpness parameter α_k that changes over training iterations:

$$c_i = \tanh(\alpha_k(r_i - \tau)).$$

For example, α_k can be gradually increased during training so that the controller behaves more conservatively in early stages and becomes more selective after the learner has stabilized. In this work, unless otherwise specified, we use a fixed α for clarity and leave dynamic controller scheduling as an extension.

Overall, the elastic critical controller transforms the evaluator’s reliability judgment into a continuous learning control signal. Reliable samples contribute positive learning signals, uncertain samples are down-weighted or suppressed, and highly unreliable samples can be discouraged from influencing the learner in the direction of their observed labels.

1.4 Gold-Guided Learning and Unlearning

After the gold-guided reference training stage, the learner is initialized as

$$\theta_0 \leftarrow \theta_G,$$

while the frozen reference evaluator $M_{\bar{\theta}_G}$ is used only to compute the gold-consistency score r_i and the corresponding controller c_i .

For a mixed training sample $(x_i, y_i) \in D_t$, we denote its supervised loss under the current learner as

$$\ell_i(\theta_k) = \ell(M_{\theta_k}(x_i), y_i).$$

In standard supervised learning, the model update on a mini-batch $B \subset D_t$ is given by

$$\theta_{k+1} = \theta_k - \eta \frac{1}{|B|} \sum_{i \in B} \nabla_{\theta} \ell_i(\theta_k),$$

where η is the global learning rate.

In our proposed gold-guided critical learning framework, each sample gradient is modulated by its elastic controller c_i . The controlled mixed-data update becomes

$$\theta_{k+1} = \theta_k - \eta \underbrace{\frac{1}{|B|} \sum_{i \in B} c_i \nabla_{\theta} \ell_i(\theta_k)}_{\text{controlled mixed-data learning}}.$$

Here, c_i should be understood as a sample-wise gradient controller rather than an optimizer-level learning rate. The optimizer still uses the shared global learning rate η , while c_i controls the contribution and direction of each sample-specific gradient within the mini-batch. Equivalently, for an individual sample, c_i induces an effective update scale ηc_i , but the actual formulation is expressed as gradient modulation because different samples in the same mini-batch may have different reliability scores and therefore different controller values.

When c_i is positive, the model learns from the sample in the usual gradient descent direction. A larger positive value indicates that the observed training pair (x_i, y_i) is more consistent with the gold-guided reliability signal and therefore contributes more strongly to learning. When c_i is close to zero, the sample has little influence on the parameter update, which reduces the effect of uncertain training signals. When c_i is negative, the gradient direction is reversed, discouraging the learner from fitting the observed label. This negative control is interpreted as an anti-fitting signal against unreliable supervision, rather than as explicit label correction.

Therefore, the proposed update does not simply remove suspicious samples from the training set. Instead, it continuously controls how each observed training signal affects the learner. Reliable samples contribute positive gradients, uncertain samples are suppressed, and highly unreliable samples can be discouraged from driving the model toward their observed labels.

1.5 Gold Stability Loss

During mixed-data training, the learner M_{θ_k} is updated using the controlled gradients from the mixed dataset D_t . Although the critical controller c_i reduces the influence of unreliable samples, the learner may still gradually drift away from the reliable knowledge learned from the gold dataset, especially when the mixed dataset contains many noisy or harmful labels.

To stabilize the learner, we introduce a gold stability loss. The key idea is to use the gold dataset D_G as a reliable anchor during mixed-data training. Since D_G contains high-quality labels, the current learner should continue to perform well on D_G while adapting to the mixed dataset. Therefore, this term is designed to preserve the learner’s stability with respect to reliable gold examples, rather than to impose a generic parameter regularization such as an L_1 or L_2 penalty.

To distinguish this term from the reference training objective in Section 1.1, we define the gold stability loss as

$$L_{\text{stab}}(\theta_k; D_G) = \frac{1}{|D_G|} \sum_{(x_i^G, y_i^G) \in D_G} \ell(M_{\theta_k}(x_i^G), y_i^G).$$

Here, $L_{\text{stab}}(\theta_k; D_G)$ is computed using the current learner M_{θ_k} , not the frozen gold reference evaluator $M_{\hat{\theta}_G}$. The frozen reference evaluator is only used to compute the gold-consistency score r_i and the controller c_i . In contrast, the gold stability loss is applied to the learner that is being updated.

A small value of $L_{\text{stab}}(\theta_k; D_G)$ indicates that the current learner still preserves the reliable knowledge learned from the gold dataset. In contrast, an increasing value of $L_{\text{stab}}(\theta_k; D_G)$ suggests that the learner may be drifting away from the gold knowledge while adapting to the mixed dataset. Therefore, minimizing $L_{\text{stab}}(\theta_k; D_G)$ helps keep the learner stable with respect to the reliable gold examples.

In practice, computing $L_{\text{stab}}(\theta_k; D_G)$ over the full gold dataset at every training iteration may be expensive. Therefore, the gold stability loss can be estimated using a gold mini-batch $B_G \subset D_G$:

$$L_{\text{stab}}(\theta_k; B_G) = \frac{1}{|B_G|} \sum_{j \in B_G} \ell(M_{\theta_k}(x_j^G), y_j^G).$$

The final update direction combines the controlled mixed-data gradient and the mini-batch gold stability gradient:

$$g_k = \underbrace{\frac{1}{|B|} \sum_{i \in B} c_i \nabla_{\theta} \ell_i(\theta_k)}_{\text{controlled mixed-data learning}} + \underbrace{\lambda_G \nabla_{\theta} L_{\text{stab}}(\theta_k; B_G)}_{\text{mini-batch gold stability loss}},$$

where $B \subset D_t$ is a mixed mini-batch, $B_G \subset D_G$ is a gold mini-batch, and λ_G controls the strength of the gold stability constraint. The learner is then updated by

$$\theta_{k+1} = \theta_k - \eta g_k.$$

This update contains two complementary forces. The first term allows the learner to adapt to the mixed dataset, while each sample’s influence is controlled by c_i . The second term encourages the learner to maintain good performance on the reliable gold dataset whenever its gold-data performance begins to degrade. Therefore, the gold stability loss prevents the learner from drifting too far away from reliable gold knowledge while it learns selectively from the mixed dataset.

Thus, the gold stability loss does not replace mixed-data learning. Instead, it provides a reliable gold-data stability constraint that helps the learner preserve high-quality knowledge while learning selectively from the mixed dataset.

1.6 Discussion

The proposed formulation provides a gold-guided mechanism for controlling how the learner responds to each observed training signal in the mixed dataset. Instead of treating the mixed dataset as uniformly reliable, the method first evaluates each sample through a frozen gold reference evaluator and then converts this evaluation into a continuous sample-wise learning control signal.

A central component of this framework is the gold-consistency score r_i . This score is not intended to infer a corrected label for x_i . Instead, it measures how reliable the observed training pair (x_i, y_i) appears with respect to the knowledge learned from the gold dataset. By combining prediction-level consistency and representation-level consistency, r_i evaluates both whether the observed label is supported by the frozen evaluator’s predictive distribution and whether the sample representation is close to the reliable gold prototype of the same observed class.

The elastic controller c_i then transforms this reliability score into a sample-wise gradient control coefficient. Reliable samples produce positive learning signals, uncertain samples are down-weighted

or suppressed, and highly unreliable samples can be discouraged from pushing the learner toward their observed labels. Therefore, the method does not simply keep or discard samples. Rather, it continuously controls the strength and direction of each sample’s influence on the learner.

The gold dataset plays three related roles in this framework. First, it provides the initial parameter $\theta_0 \leftarrow \theta_G$, allowing the learner to start from a reliable knowledge state. Second, it provides a frozen reference evaluator $M_{\bar{\theta}_G}$, which estimates the gold-consistency score of each mixed training sample. Third, it serves as a stability anchor through the gold stability loss $L_{\text{stab}}(\theta_k; D_G)$, preventing the current learner from drifting away from reliable knowledge during mixed-data training.

The overall learning process is therefore guided by both sample-level criticality and global gold-data stability. The sample-level controller determines how the learner should respond to each mixed training example, while the gold stability term helps preserve reliable knowledge throughout training. This shifts the focus from simply identifying whether a sample is clean or noisy to controlling how the model should learn from, suppress, or resist each observed training signal.

This formulation should be viewed as a noise influence reduction framework rather than an explicit label correction method. The current goal is to reduce the effect of contaminated labels on the learner while preserving useful information from reliable samples. Explicit label correction can be considered as a separate extension in future work.

Algorithm 1 Gold-Guided Critical Learning

Require: Gold dataset D_G , mixed dataset D_t , model M_θ , learning rate η , reliability threshold τ , controller sharpness α , consistency weight β , gold regularization weight λ_G , total iterations K

Ensure: Updated learner parameter θ_K

- 1: Train the model on D_G by minimizing $L_{\text{ref}}(\theta; D_G)$ to obtain θ_G
- 2: Initialize learner: $\theta_0 \leftarrow \theta_G$
- 3: Freeze gold reference evaluator: $\bar{\theta}_G \leftarrow \theta_G$
- 4: Compute gold feature prototypes $\{\mu_y^G\}$ for all classes using $M_{\bar{\theta}_G}$
- 5: **for** $k = 0, 1, \dots, K - 1$ **do**
- 6: Sample mixed mini-batch $B \subset D_t$ and gold mini-batch $B_G \subset D_G$
- 7: **for** each $(x_i, y_i) \in B$ **do**
- 8: Compute r_i^{prob} , r_i^{sim} , and r_i using the gold-consistency score formulation
- 9: Compute controller:

$$c_i \leftarrow \tanh(\alpha(r_i - \tau))$$

- 10: Define mixed-sample loss:

$$\ell_i(\theta_k) \leftarrow \ell(M_{\theta_k}(x_i), y_i)$$

- 11: **end for**
- 12: Estimate gold stability loss on B_G :

$$L_{\text{stab}}(\theta_k; B_G) = \frac{1}{|B_G|} \sum_{j \in B_G} \ell(M_{\theta_k}(x_j^G), y_j^G)$$

- 13: Compute controlled update direction:

$$g_k \leftarrow \frac{1}{|B|} \sum_{i \in B} c_i \nabla_{\theta} \ell_i(\theta_k) + \lambda_G \nabla_{\theta} L_{\text{stab}}(\theta_k; B_G)$$

- 14: Update learner:

$$\theta_{k+1} \leftarrow \theta_k - \eta g_k$$

- 15: **end for**
 - 16: **return** θ_K
-

2 Positioning: From Sample-Centered Reweighting to Evaluator-Centered Control

Existing noisy-label learning and continual learning methods often follow a sample-centered perspective. In this view, each training example is first assigned a sample quality score, and this score is then used to determine whether the example should be kept, removed, or reweighted:

$$(x_i, y_i) \rightarrow \text{sample quality score} \rightarrow \text{keep/drop/reweight}.$$

Such methods mainly focus on the property of the sample itself. For example, they may ask whether the sample has a small loss, whether the sample is likely to be clean, whether the sample should be stored in a memory buffer, or whether the observed label should be corrected. Although

these strategies are effective for reducing the influence of noisy data, they are primarily designed as data-processing or sample-selection mechanisms.

In contrast, our proposed Critical Learning framework adopts an evaluator-centered perspective. Instead of only estimating whether a sample is clean or noisy, the evaluator determines how the learner should respond to each training signal. In this framework, the frozen gold reference evaluator $M_{\bar{\theta}_G}$ produces a gold-consistency score, which is then transformed into a critical controller:

$$(M_{\bar{\theta}_G}, x_i, y_i) \rightarrow r_i \rightarrow c_i \rightarrow \text{learning controller}.$$

Here, the evaluator is not merely a data-cleaning module. Rather, it directly controls the learning dynamics by modulating the sample-wise gradient contribution during training.

The resulting update rule can be written as

$$\theta_{k+1} = \theta_k - \eta \left[\underbrace{\frac{1}{|B|} \sum_{i \in B} c_i \nabla_{\theta} \ell_i(\theta_k)}_{\text{evaluator-controlled mixed-data learning}} + \underbrace{\lambda_G \nabla_{\theta} L_{\text{stab}}(\theta_k; D_G)}_{\text{gold stability loss}} \right],$$

where

$$\ell_i(\theta_k) = \ell(M_{\theta_k}(x_i), y_i).$$

Here, $\ell_i(\theta_k)$ is the supervised loss for the mixed training sample (x_i, y_i) , $L_{\text{stab}}(\theta_k; D_G)$ is the gold stability loss of the current learner, and c_i is the evaluator-induced critical controller.

The controller c_i represents the model’s response to the corresponding training signal:

$$c_i > 0 \Rightarrow \text{learning},$$

$$c_i \approx 0 \Rightarrow \text{suppression},$$

$$c_i < 0 \Rightarrow \text{unlearning-like correction}.$$

Therefore, the goal is not simply to decide whether a sample should be retained or discarded. Instead, each training signal is first judged by a frozen gold reference evaluator and then transformed into a learning control signal. This shifts the focus from sample management to learning behavior control.

This distinction makes Critical Learning closer to a training paradigm than a conventional sample reweighting strategy. While sample-centered methods ask whether a data point is reliable, evaluator-centered Critical Learning asks how the model should learn from, ignore, or correct the influence of a given training signal.

#	Train	Test	Scene meaning	Why run it	Priority
1	Noisy Label	Clean Feature and Label	Anti-noise learning ability under corrupted labels	Can the controller recover correct boundaries despite mislabeled data?	Core
2	Noise Feature	Clean Feature and Label	Robustness to corrupted inputs during training	Does the model learn clean structure when inputs are degraded but labels correct?	Core
3	Noise Feature and Label	Clean Feature and Label	Combined-corruption learning ability	Hardest training case; survives simultaneous label and feature noise.	Core
4	Clean Feature and Label	Noisy Feature	Pure generalization robustness to input shift	Baseline: a cleanly trained model meets degraded inputs at test.	Core
5	Noise Feature	Noisy Feature	Matched train/test feature corruption	Does train-time feature noise improve test-time robustness? Pairs with Scene 4.	Core
6	Noise Label	Noisy Feature	Cross-axis stress test	Does a label-noise-trained model still generalize under input shift?	Opt.
7	Noise Feature and Label	Noisy Feature	Worst-case combined stress test	Both training problems plus test-time shift; checks for catastrophic break.	Opt.

Table 1: Experimental design across seven train–test noise scenes. Test labels are clean in all scenes, so Top-1 Accuracy is a valid quality measure throughout. Scenes 1–3 isolate *learning ability* under dirty training data; Scenes 4–7 isolate *generalization robustness* under corrupted test inputs.

3 Experiments

3.1 Data

We prepare to validate our method in two different categories: pictures for CV and Text for NLP.

- Pictures: STL_10, 96 * 96 pixel, 10 classes, with 13K samples
- Flower-102, 500 * 400 Pixels, 102 flower categories
- AG News, 4 categories including World, Sport, Business, and Tech

3.2 Model

We use a compact three-layer convolutional neural network (CNN) as the primary learner in our experiments. The purpose of this model choice is not to maximize benchmark accuracy, but to provide a simple and controlled backbone for evaluating the effect of the proposed gold-guided critical learning mechanism. A lightweight model also makes it easier to observe how noisy labels and corrupted test conditions affect the learning dynamics, since the performance changes are less likely to be hidden by excessive model capacity.

The CNN consists of three convolutional blocks followed by a fully connected classifier. Each convolutional block contains a convolutional layer, a nonlinear activation function, and a pooling operation. The final feature representation is flattened and passed to a linear classification head. For image datasets, the number of output units is set according to the number of classes in the corresponding dataset.

Using this simple backbone allows us to focus on the behavior of the proposed controller rather than on architectural complexity. In particular, we aim to examine whether the gold-guided evaluator can suppress misleading training signals, preserve useful learning directions, and maintain stable performance under different train–test noise conditions.

For additional robustness evaluation, the same training framework can also be applied to a stronger backbone such as ResNet-18. This extension is useful for examining whether the proposed learning mechanism remains effective when the learner has higher representation capacity.

4 Evaluation Metrics

We evaluate model performance across seven train–test noise scenes. Following Table 2, Top-1 Accuracy is used as the headline performance metric in every scene because the test labels remain clean. The auxiliary metric is chosen according to the dominant failure mode of each scene. For training corruption, we emphasize memorization and overfitting diagnostics. For test-time feature corruption, we emphasize calibration and uncertainty-based robustness diagnostics. In addition, Gradient Norm is reported across all scenes as a universal stability indicator.

4.1 Metrics for Clean Test Settings (Scenes 1–3)

Scenes 1–3 evaluate whether the model can learn reliable decision boundaries when the training data are corrupted but the test inputs and test labels remain clean. Therefore, Top-1 Accuracy directly measures the final prediction quality, while the second metric diagnoses how the model responds to corrupted training signals.

Top-1 Accuracy. The fraction of test samples whose top predicted class matches the ground-truth label:

$$\text{Acc@1} = \frac{1}{N} \sum_{i=1}^N \mathbf{1}[\hat{y}_i^{(1)} = y_i]. \quad (1)$$

Since the test labels are clean, Top-1 Accuracy is a valid measure of generalization quality. In scenes with corrupted training labels or features, a robust method should maintain high clean-test accuracy despite misleading training signals.

Forgetting Events. A forgetting event for sample i occurs when the model classifies it correctly at epoch $t - 1$ but incorrectly at epoch t :

$$F_i = \sum_{t=2}^T \mathbf{1}[\hat{y}_i^{(t-1)} = y_i \wedge \hat{y}_i^{(t)} \neq y_i]. \quad (2)$$

Forgetting Events are especially useful for label-corrupted training scenes. Samples affected by noisy labels tend to be learned and forgotten more frequently, reflecting unstable memorization behavior. Therefore, we use Forgetting Events as the auxiliary metric for Scene 1, where the training labels are corrupted, and as part of the auxiliary metric for Scene 3, where both labels and features are corrupted.

Train/Test Loss Gap. The per-epoch difference between test loss and training loss:

$$\Delta L(t) = L_{\text{test}}(t) - L_{\text{train}}(t). \quad (3)$$

A widening gap indicates that the model is fitting the training data more strongly than it generalizes to clean test data. This is particularly important when training features are corrupted, because the model may overfit degraded or spurious input patterns. Therefore, we use the Train/Test Loss Gap as the auxiliary metric for Scene 2 and combine it with Forgetting Events in Scene 3.

4.2 Metrics for Feature-Corrupted Test Settings (Scenes 4–7)

Scenes 4–7 evaluate robustness when the test inputs are corrupted while the test labels remain clean. Top-1 Accuracy is still valid because the ground-truth labels are not changed. However, accuracy alone may not fully capture whether the model becomes overconfident, uncertain, or unstable under degraded test inputs. We therefore complement accuracy with calibration and uncertainty-based metrics.

Expected Calibration Error (ECE). ECE partitions predictions into M confidence bins and measures the weighted average gap between confidence and accuracy:

$$\text{ECE} = \sum_{m=1}^M \frac{|B_m|}{N} |\text{acc}(B_m) - \text{conf}(B_m)|, \quad (4)$$

where $\text{acc}(B_m)$ and $\text{conf}(B_m)$ are the mean accuracy and mean confidence within bin B_m , respectively. Under feature corruption, a brittle model may become overconfident on degraded inputs, producing high-confidence errors. A robust model should maintain low and stable ECE. We therefore use ECE for feature-corrupted test scenes where calibration is a central concern, especially Scenes 4, 5, and 7.

Prediction Entropy. The mean Shannon entropy of the softmax output over the test set:

$$\bar{H}(t) = -\frac{1}{N} \sum_{i=1}^N \sum_{c=1}^C p_{\theta}(c | x_i) \log p_{\theta}(c | x_i), \quad (5)$$

where C is the number of classes. Prediction Entropy measures the uncertainty of the model’s output distribution. Under feature corruption, rising entropy indicates that the model becomes increasingly uncertain when test inputs are degraded, while stable entropy suggests more consistent prediction behavior. We use Prediction Entropy for Scenes 4, 6, and 7, where test-time feature corruption is expected to induce uncertainty.

#	Train	Test	Metric 1	Metric 2
1	Label	Clean	Top-1 Acc	Forgetting Events
2	Feature	Clean	Top-1 Acc	Train/Test Loss Gap
3	Hybrid	Clean	Top-1 Acc	Forgetting Events + Loss Gap
4	Clean	Feature	Top-1 Acc	ECE / Entropy
5	Feature	Feature	Top-1 Acc	ECE
6	Label	Feature	Top-1 Acc	Prediction Entropy
7	Hybrid	Feature	Top-1 Acc	ECE + Entropy

Table 2: Metric choices per scene. Top-1 Accuracy is the headline measure in every scene (clean test labels). The second metric diagnoses the dominant failure mode: memorization (Forgetting Events) and overfitting (Loss Gap) for training corruption; calibration/uncertainty (ECE, Entropy) for test-time feature corruption. Gradient Norm is reported across all scenes as a universal stability signal.

4.3 Universal Metric (All Settings)

Gradient Norm. The ℓ_2 norm of the loss gradient with respect to all parameters:

$$\|\nabla\mathcal{L}\|(t) = \left\| \frac{\partial\mathcal{L}}{\partial\theta} \right\|_2 \bigg|_{\text{epoch } t}. \quad (6)$$

Noisy labels and corrupted features introduce unstable or conflicting training signals, which can manifest as high-variance gradient norms. A noise-robust method should produce smoother and more stable gradient norms across all seven scenes. Therefore, Gradient Norm is reported as a universal stability metric in addition to the scene-specific metrics in Table 2.